

微服务架构下分布式定向日志采集组件的设计与实现

石洪雷*, 赵涓涓

太原理工大学, 山西太原, 030024

摘要

微服务架构下, 日志来源高度分散、格式各异、总量庞大, 给节点资源、网络带宽和集中存储都带来了很大压力。本文提出一种分布式定向日志采集框架与原型组件, 核心思路是“只采必要日志”。框架分为三层: 网关预筛选、节点定向采集和中心二次过滤。日志中心根据网关流量日志动态生成关注清单, 驱动各节点按清单定向采集应用日志。节点侧采用固定缓存块算法和压力感知指数退避机制, 在资源受限时保障传输可靠性。三层之间通过定向策略三次转换算法保持策略语义一致。原型基于 Go 语言和 Grafana Loki 实现, 应用日志采用 Apache Combined 风格并与网关 URL 对齐。实验在 WSL/Docker 八节点集群上进行对比测试 (180s、并发 50、每模式重复 3 次), 并补充了十六/三十二节点规模复核、策略仿真消融、基于真实分布的多进程模拟与算法微基准测试。结果显示, 定向采集将 Loki 入库量控制在 72 条, 而同架构全量基线为 4388 ± 1 条, 降幅达 98.4% (该降幅包含定向过滤与发射频率差异的联合效果; 多进程同频模拟表明策略过滤的独立降幅为 67.8%)。清空统计后的规模复核中, 十六节点 Loki 入库为 48.0 ± 5.2 条/轮, 三十二节点稳定在 99.0 ± 0.0 条/轮。Agent 平均 CPU 与内存分别降低 37.5% 和 2.0% (绝对值均较低)。端到端链路 (Nginx 共享内存 $\rightarrow$ Agent gRPC $\rightarrow$ Redis $\rightarrow$ 关注清单生成) 已实测打通。关注清单生成算法在 5000 条日志上耗时约 15 ms, 指数退避机制通过 `agent/pkg/retry/backoff_test.go` 全部单元测试。

关键词: 微服务; 分布式日志采集; 定向过滤; 动态策略; 指数退避; Grafana Loki

中图法分类号: TP393 **文献标识码:** A

建议引用格式: 石洪雷, 赵涓涓. 微服务架构下分布式定向日志采集组件的设计与实现 [J]. 软件学报投稿稿, 2026.

Design and Implementation of a Distributed Directed Log Collection
Component in Microservice Architecture

*通讯作者, E-mail: shihonglei0042@link.tyut.edu.cn

Abstract: Microservice architectures scatter log sources across many independently deployed services, each emitting logs in heterogeneous formats whose aggregate volume grows linearly with request concurrency. The resulting overhead on node CPU, network bandwidth, and centralized storage motivates a shift from full-volume collection to selective, traffic-aware ingestion. This paper proposes a distributed directed log collection framework whose guiding principle is to “collect only what matters.” The framework comprises three layers: gateway pre-filtering, node-side directed collection, and center-side secondary filtering. A log center dynamically generates an attention list from gateway traffic logs (URLs, status codes, response latencies) and distributes it to each node, which then collects only the application logs matching the list. Fixed-size cache blocks and pressure-aware exponential backoff maintain transmission reliability under resource constraints, while a triple strategy transformation keeps policy semantics consistent across all three layers. A prototype built with Go and Grafana Loki is evaluated on a WSL/Docker cluster. In eight-node comparison experiments (180s, concurrency 50, three repeats per mode), directed collection reduces Loki-ingested logs to 72 entries versus 4388 ± 1 for the same-architecture full-collection baseline (98.4% reduction, combining directed filtering and emission-rate differences; a same-frequency multi-process simulation isolates a 67.8% independent reduction from policy filtering alone). Scale validations after counter reset show 48.0 ± 5.2 ingested entries per round at 16 nodes and a stable 99.0 ± 0.0 at 32 nodes. Mean agent CPU and memory overhead decrease by 37.5% and 2.0%, respectively, with absolute values remaining below 0.1%. The end-to-end pipeline, from Nginx shared memory through agent gRPC upload, Redis caching, to attention-list generation, is verified in cluster experiments. Attention-list generation completes in approximately 15ms on 5000 log entries, and the exponential backoff module passes all unit tests in `agent/pkg/retry/backoff_test.go`. These results suggest that gateway-driven directed collection can substantially reduce log ingestion volume at the collection front end without modifying application code.

Key words: microservice; distributed directed log collection; attention list; dynamic policy; exponential backoff; Grafana Loki

Authors: Honglei Shi (Corresponding author), Juanjuan Zhao, Taiyuan University of Technology, Taiyuan 030024, China. E-mail: shihonglei0042@link.tyut.edu.cn

1 引言

微服务架构把一个大系统拆成多个独立服务，带来了更好的可扩展性和开发灵活性。容器编排、Sidecar 代理和服务治理等技术的成熟，使云原生应用的大规模部署成

为现实^[1-6]。但拆分也带来一个棘手问题——日志高度分散。每个服务实例各自输出日志，格式五花八门，总量随并发请求线性增长。传统做法是“全量采集”，先把日志统统收上来再说。这在中小规模场景下还能接受，一旦集群达到数十甚至数百节点，问题就暴露了：节点 CPU 和内存被持续占用，南北向带宽成为瓶颈，存储与索引成本快速攀升，而且大量低价值日志反而把真正有用的故障信息给淹没了^[7-10]。

目前的研究主要集中在日志存储压缩、采样过滤、故障诊断和 eBPF 无侵入采集等方向，但很少有人从**网关流量感知**的角度出发，动态驱动各节点的应用日志定向采集。包航宇等^[11]总结了智能运维的实践现状与标准化框架，贾统等^[12]系统梳理了基于日志的分布式故障诊断技术。本文的思路不同，把关注点前移到**采集前端**——网关是南北向流量的入口，天然掌握着 URL、状态码、响应时延等结构化信号。利用这些信号，不需要侵入业务代码，就能识别出哪些请求是高价值的（比如出错的、响应慢的），然后指导下游节点只采集与这些请求相关的应用日志，做到“只采必要日志”。

本文要回答的核心问题是：在不侵入业务代码、不改变微服务调用链的前提下，能不能把网关层观测到的流量异常信号，转化为节点侧的日志采集约束？也就是说，既保留异常和慢请求的高价值上下文，又能大幅降低日志入库量、节点资源开销和后端存储压力。围绕这个问题，本文拆解出四个可验证的子问题：（1）网关流量日志能不能动态生成有效的关注清单？（2）关注清单能不能在网关、节点和中心三层之间保持语义一致？（3）定向采集会不会给 Sidecar 带来不可接受的资源开销？（4）固定缓存块和压力感知退避机制能不能在中心不可达或短时拥塞时保障可靠传输？

本文提出以下科学假设：（H1）网关访问日志中的 URL、状态码和响应时延可以作为应用日志价值的先验信号，足以生成覆盖异常与慢请求模式的动态关注清单。（H2）把这份关注清单下发给节点侧 Sidecar 后，能在规则级保留关键请求上下文，同时使 Loki 入库量相比同架构全量采集显著下降。（H3）通过网关预筛选、节点定向采集和中心二次过滤之间的三次策略转换，可以维持跨层策略语义一致，形成端到端的采集闭环。（H4）固定缓存块与指数退避机制能够在低资源占用下提供有界重试和传输缓冲能力。后文实验部分以 RQ1-RQ4 对上述假设逐一验证。

基于上述问题与假设，本文提出**分布式定向日志采集框架与原型组件**。框架分为三层：**网关预筛选、节点定向采集、中心二次过滤**，三层之间通过三次策略转换保持语义一致。和 Promtail/Fluent Bit “全量推送后在存储端压缩”的传统方案相比，本文直接在采集前端就把 Loki 入库量控制在百条量级（八节点实测详见 §6.3），从源头降低了网络和存储开销。主要贡献如下：

（1）提出一个可复用的三层分布式定向采集框架。网关节点和微服务节点均以 Sidecar 方式部署，中心负责策略生成与二次过滤，节点负责本地匹配、缓存与可靠上传。这个框架补上了 AIOps 数据链路中“采集前端减量”这一缺失环节。

（2）给出关注清单动态生成、固定缓存块、定向策略三次转换和压力感知指数退避四个关键机制的形式化描述与 Go 语言实现。其中，关注清单生成的排序实现复杂度上界为 $O(N \log N)$ ，最小堆实现为 $O(N \log K)$ （ K 为清单大小， $K \ll N$ ；见算法 1）。

(3) 构建基于 Docker Compose 的可复现原型和多进程模拟器。在八节点集群上完成 180s、 $n=3$ 的重复对比实验、算法微基准测试与系统级组件消融，并补充了清空统计后的十六/三十二节点规模复核。八节点实测结果显示，定向采集的 Loki 入库量为 72 条，而全量基线为 4388 ± 1 条（该降幅包含定向过滤与发射频率差异的联合效果）。多进程同频模拟进一步证实，策略过滤本身的独立降幅为 67.8%。十六/三十二节点复核则验证了定向模式在更大规模下依然能保持低入库量和低 Agent CPU 开销。

本文第 2 节综述相关工作；第 3 节介绍系统总体设计；第 4 节阐述关键算法；第 5 节描述实现细节；第 6 节给出实验与分析；第 7 节总结全文。

2 相关工作

2.1 集中式日志栈

ELK/EFK 以 Elasticsearch 为核心，检索能力强，但索引成本高昂。多项日志管理综述指出，大规模系统的日志在采集、传输、存储、解析和查询各阶段均会产生显著开销^[7-9;13]。为此，已有研究提出日志压缩、模式挖掘、模板解析、云端低成本存储和离线数据集构建等方法^[14-17]。然而，上述工作大多聚焦于日志入库之后的压缩与治理，默认日志已被全量采集到中心端。与之不同，本文关注的是日志进入中心之前的**采集前端定向减量**。

2.2 采样与追踪关联

OpenTelemetry 尾部采样依赖分布式追踪上下文，在请求完成后决定是否保留相关 span 或日志^[18]。多项调研表明，追踪、指标和日志三类遥测数据在微服务排障中需要联合分析^[10;19;20]。国内研究则围绕分布式追踪、调用链存储、服务依赖发现与微服务故障检测等方向展开^[21-26]。在根因定位方面，已有工作分别从轨迹监测、拓扑依赖、多源信号和方法体系等角度推进后端分析^[27-30]。与上述工作不同，本文不以已完整采集的 trace/log 数据为前提，而是利用网关 access log 动态生成 URL 模式清单，在 Sidecar 层即决定哪些应用日志需要上传。

2.3 边缘采集与可靠传输

边缘计算和云原生场景对采集组件提出了严苛要求：在低带宽、低资源或中心暂时不可达时，组件需保持有界退化^[10;31;32]。此外，Service Mesh 研究表明 Sidecar 自身也会引入额外延迟和资源消耗^[33;34]，因此采集逻辑必须严格控制 CPU、内存和网络占用。为此，本文在 Sidecar 中引入固定上限缓存块与压力感知退避机制，并辅以 BoltDB 本地兜底，确保节点侧采集在资源受限条件下依然可控。

2.4 AIOps 实践与标准化

包航宇等^[11] 基于大规模企业调研，总结了智能运维的实践现状，并提出了 AIOps-OSA 能力建设框架。后续研究进一步表明，生产系统需要将运行时数据、自动化分析和工程治理形成闭环^[20;35;36]。本文聚焦其中**数据采集能力**的前端减量环节：借助网关流量驱动的关注清单，从源头降低进入日志平台和后续分析模块的数据规模。

2.5 日志解析与故障诊断

贾统等^[12] 聚焦日志收集之后的下游环节，系统梳理了日志解析、异常检测、故障定位和知识提取等技术。在日志解析方面，Logram、模板识别及大规模日志解析评测等工作推动了非结构化日志向结构化事件的转换^[17;37;38]。在异常检测方面，CNN-text、LogFormer、机器学习综述、深度学习失效预测、融合学习时序检测以及微服务性能异常检测等研究展示了日志智能分析的最新进展^[39-45]。此外，多变量日志异常检测与图式根因分析进一步扩展了故障诊断的能力边界^[46;47]。与上述工作相比，本文关注的是**诊断的前置条件**——在日志尚未大规模入库之前完成定向减量，同时尽量保留错误、慢请求等高价值上下文。

2.6 微服务工程与云原生生态

微服务架构涉及服务拆分、持续工程治理与运行时配置管理等多个方面^[3;48-50]。容器编排和微服务设计模式为 Sidecar 部署提供了基础设施支撑^[1;2;4]。鉴于此，本文在实验中明确区分了三类证据：原型集群实测、基于真实分布的多进程模拟验证，以及生产级证据。在基线选取上，本文将 Promtail 静态过滤作为同负载模拟基线，将 OpenTelemetry 和 eBPF 列为后续端到端部署基线。

2.7 新兴采集技术与最接近先例

eBPF 技术使内核级日志与事件采集成为可能^[34;51]。然而，eBPF 侧重**基础设施级**事件，对**应用级**日志的定向采集支持有限，且缺乏网关流量驱动的动态策略源。OpenTelemetry 尾部采样以**分布式追踪 span** 为单位做取舍，与本文以 **URL 模式**为粒度的应用日志定向过滤构成互补关系。

与本文最接近的工业实践有三类。其一，Envoy 代理的 AccessLog 过滤器支持按路由或状态码配置日志输出策略，但仅作用于代理自身的 access log，无法驱动下游微服务的应用日志采集。其二，Kong API 网关的 File-log/TCP-log 插件可按路由启用日志记录，但属于静态配置，不具备基于流量统计动态生成 URL 模式的能力。其三，OpenResty 生态的 lua-resty-logger-socket 等模块实现了 Lua 层的动态日志控制，但策略局限于网关节点自身，未形成跨节点的采集闭环。

上述先例存在两个共同局限：日志过滤决策**局限于单一节点**（代理或网关），且多依赖**静态或手工配置**规则。本文的增量创新在于：以网关流量统计为输入，自动生成 Top- K URL 模式清单，经 gRPC 下发至多个微服务节点驱动应用日志定向采集，最后由中心二次过滤保障语义一致。这一流程形成了“网关流量 → 多节点应用日志动态清单 → 中心二次过滤”的**贯通闭环**，以规则驱动的轻量匹配替代 ML 推理，更适合边缘资源受限环境。

2.8 与代表性方案对比

表 1 从策略输入、减量位置、日志削减证据、动态策略来源和 Sidecar 开销等维度，将本文方案与代表性方案进行对比。

表 1: 与代表性日志采集/可观测性方案对比

方案	策略驱动源	减量位置	日志削减证据	动态策略来源	Sidecar 开销	本文关系
ELK/EFK 全量	静态配置	存储后端	通常无采集前减量	无	依部署而定	存储与索引成本高
Loki/Promtail	标签/级别	静态过滤	依规则配置	静态文件	低至中	工业参照
OTel 尾部采样 ^[18]	Trace 上下文	Span 保留	依采样策略	追踪上下文	采集器开销	URL 粒度互补
eBPF 日志 ^[34;51]	内核探针	内核/系统事件	依探针选择	内核事件	无应用 Sidecar	零侵入但无网关策略驱动
AIOps 标准化 ^[11]	运维能力框架	平台能力层	非采集算法	标准化能力模型	非组件级	本文支撑数据采集能力
日志故障诊断综述 ^[12]	已入库日志	分析阶段	非采集减量	后端诊断任务	非组件级	本文减少诊断输入规模
调用链异常定位 ^[24]	调用链	定位阶段	非采集减量	Trace/控制流	依追踪系统	侧重事后定位
本文	网关流量	采集前端	72 vs 4388 条 *	Top- K URL 清单	0.05% CPU*	前端定向减量框架

2.9 本文定位

综上所述，包航宇等的 AIOps 标准化研究着力解决平台能力建设问题^[11]，贾统等的日志诊断综述着力解决后端分析方法体系问题^[12]。二者均需要稳定、低成本且高价值的日志输入作为基础。本文定位为 **AIOps 采集前端的定向减量技术**：将网关 access log 作为动态策略输入，下发至各微服务节点执行应用日志定向采集，再由中心二次过滤后入库。借助关注清单、三次策略转换与可复现的 Docker 原型，本文在采集阶段将入库量控制在百条量级 *，并与全量 Sidecar 架构进行了对照实测（详见 §6.3 表 5）。

* 八节点、180s、 $n=3$ ；全量基线发射频率（400ms）高于定向模式（2s），降幅为策略过滤与源配置联合效果；基于真实分布的多进程同频模拟表明，策略过滤独立降幅为 67.8%（§6.3）。

3 系统总体设计

3.1 架构概述

系统由两个核心角色组成：**日志节点采集器 (Agent)** 和 **日志中心 (Center)**。Agent 有两种部署形态——网关节点实例和微服务节点实例。网关实例通过 OpenResty Lua 采集 HTTP 流量日志，微服务实例负责采集应用访问日志。Center 的职责包括：接收

Agent 上传的日志、生成关注清单、下发规则、执行二次过滤，最终将结果写入 Loki。总体架构见图 1。

从能力边界来看，整个框架可以分为三层：**网关预筛选**、**节点定向采集**、**中心二次过滤**，如表 2 所示。三层分别负责运行时流量感知、本地日志减量和中心入库控制，每层的输入输出都可以独立审计，也方便与 AIOps 平台的数据采集能力对接。

表 2: 分布式定向日志采集框架的三层输入/输出

层级	输入	输出	作用
网关预筛选	HTTP access log、状态码、响应时延	高价值流量日志与 URL 模式候选	从入口流量识别错误、慢请求和热点异常模式
节点定向采集	关注清单、本地 Apache Combined 风格应用日志	命中关注清单的应用日志批次	在 Sidecar 内完成采集前端减量，降低上传量
中心二次过滤	节点上传批次、关注清单版本、去重状态	Loki 入库日志与可查询标签	统一策略语义，过滤过期或重复日志，保留高价值上下文

3.2 工作流程

系统的运行过程分为以下五个步骤：

1. 网关在 `log_by_lua` 阶段采集 URL、状态码、响应时延等字段，写入共享内存缓冲区；
2. 网关 Agent 定期从缓冲区拉取流量日志，通过 gRPC 上传到 Center，Center 将其缓存到 Redis；
3. Center 对流量日志做高价值过滤和 URL 聚类，生成 Top- K 关注清单，再通过 gRPC 下发给各节点；
4. 微服务 Agent 拿到清单后，匹配本地应用日志，命中的日志写入固定缓存块，异步压缩上传；
5. Center 做二次过滤和去重，最后批量推送到 Loki，供 Grafana 查询使用。

3.3 部署模式

Sidecar 模式：Agent 与业务容器共享网络命名空间 (`network_mode: service:*`)，适用于 Kubernetes 或 Docker Compose 环境。**混合开发模式**：基础设施跑在容器里，

图1 分布式定向日志采集系统总体架构

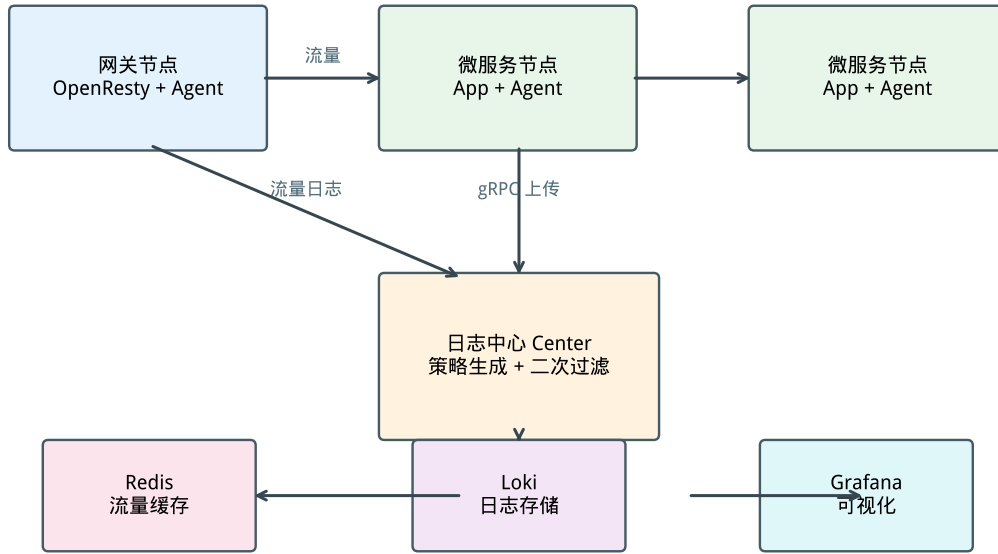


图 1: 分布式定向日志采集系统总体架构

Center 和 Agent 在本地调试。原型部署在 WSL2 上，网关映射端口 8088（避开 Windows IIS 占用的 80 端口）。

4 关键算法

4.1 关注清单动态生成

输入：时间窗口内网关流量日志集合 $L = \{l_1, \dots, l_N\}$ ；定向策略 S （响应时延阈值 T 、错误码集合 E ）。

输出：关注清单 $A = \{(p_i, w_i)\}$ ， p_i 为泛化 URL 模式， w_i 为权重。

算法的核心思路很直接：先线性扫描 N 条流量日志，把满足阈值条件的记录映射到 M 个泛化 URL 模式（ $M \leq N$ ），然后从中选出权重最高的 Top- K 项。Top- K 选取有两种实现方式：一种是全量排序，复杂度为 $O(N + M \log M)$ ，上界可简写为 $O(N \log N)$ ；另一种是维护一个大小为 K 的最小堆，复杂度为 $O(N + M \log K)$ ，当 $K \ll M$ 时效率更高。当前原型采用排序实现，主要是便于审计和复现实验。

URL 泛化规则包括：纯数字路径段映射为 $\{id\}$ ，UUID 或长哈希段映射为 $\{uuid\}$ ，查询参数按键名归一化。这样一来，Apache Combined 风格的访问日志就能直接转换为关注清单项。如果遇到加密日志、非 URL 型业务日志，或者异常极稀疏的窗口，算法会退化为仅保留 ERROR 级别的兜底规则。微基准测试显示，处理 5000 条合成日志大约耗时 15 ms（图 6）。和 Promtail 依赖静态标签配置不同，本算法以网关 access log 为

Algorithm 1: 关注清单动态生成**Input:** 流量日志集合 L , 策略 $S = (T, E, K, \text{TTL})$ **Output:** 关注清单 A

```

1  $H \leftarrow \emptyset$ ;
2 foreach  $l \in L$  do
3   if  $\text{rt}(l) > T$  or  $\text{status}(l) \in E$  then
4      $p \leftarrow \text{Generalize}(l.\text{url})$ ;           // 数字  $\rightarrow \{\text{id}\}$ , UUID  $\rightarrow \{\text{uuid}\}$ 
5      $H[p].\text{count} \leftarrow H[p].\text{count} + 1$ ;
6      $H[p].\text{severity} \leftarrow \max(H[p].\text{severity}, \text{Sev}(l))$ ;
7 foreach  $p \in H$  do
8    $w_p \leftarrow \alpha \cdot H[p].\text{count} + \beta \cdot H[p].\text{severity}$ ;
9  $A \leftarrow \text{TopK}(H, K)$ ;                      // 按  $w_p$  降序取前  $K$  项
10 foreach  $(p_i, w_i) \in A$  do
11   附加 TTL;
12 return  $A$ ;

```

策略输入，能动态识别高价值 URL 模式。

4.2 固定缓存块

每个节点预先分配一块固定大小的内存 B （默认 64–128 MB），内部组织为环形队列，同时约束条目数和字节数。当占用率超过阈值 θ （默认 80%）或定时器触发时，就异步做 gzip 压缩上传；队列满了则按 FIFO 策略淘汰最旧的条目，确保内存使用始终有界。

4.3 定向策略三次转换

为了让策略在不同层级之间保持语义一致，本文设计了三次转换机制（图 2）：

阶段	输入	输出	作用
第一次	定向策略 S	网关采集规则	Nginx 预筛选（阈值 $T/5$ ）
第二次	网关日志 + S	Agent 过滤规则	驱动节点定向采集
第三次	关注清单 + 服务日志	Loki 存储规则	中心二次过滤与入库

三次转换能正确工作，靠的是两个前提：同一关注清单版本号 v 和同一 URL 泛化函数 $\text{Generalize}(\cdot)$ 。第一次转换用一个较宽松的阈值筛选流量日志，目的是不在网关侧过早丢掉潜在异常。第二次转换把候选集合压缩为 Top- K URL 模式，让 Agent 本地的匹配规则与中心策略保持一致。第三次转换在入库前校验清单版本、TTL 和去重键，过

滤掉过期或重复的日志。这样一来，只要服务日志中的 URL 字段可以用同一泛化函数解析，而且清单 TTL 没有过期，节点侧命中的日志就不会在中心侧因为策略语义漂移而被错误丢弃。如果遇到字段缺失或格式不兼容的情况，系统会按 ERROR 级别的兜底规则处理，并在 Center 记录未匹配原因。

图2 定向策略三次转换算法流程

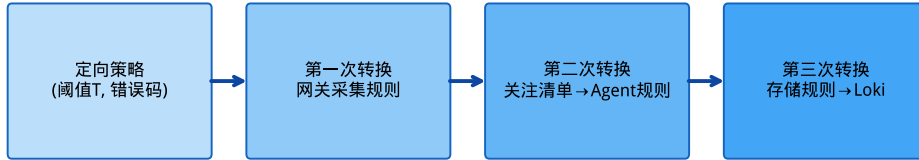


图 2: 定向策略三次转换算法流程

4.4 压力感知指数退避

上传失败时，第 n 次重试前的等待时间定义为

$$d_n = \min(d_{\max}, d_0 \rho^n + \text{rand}(0, d_0 \rho^n \xi)). \quad (1)$$

其中 n 是当前重试序号， $d_0=200\text{ ms}$ 是初始延迟， $\rho=2.0$ 是指数增长因子， $d_{\max}=30\text{ s}$ 是单次等待的上界， $\xi=0.3$ 是随机抖动系数， $\text{rand}(\cdot)$ 是均匀随机扰动。当前实现最多重试 6 次；节点资源压力大时 d_n 会翻倍；如果遇到不可重试的错误，则直接写入 BoltDB 本地兜底。延迟分布见图 3。

由于 d_n 被 $d_{\max}$ 截断，且最大重试次数固定为 6，单批日志的在线重试等待时间有明确的上界：正常压力下不超过 $6d_{\max}$ ，高压翻倍时不超过 $12d_{\max}$ 。超过重试上限后，这批日志会进入本地 BoltDB 兜底队列，后续由后台恢复任务按清单版本重新上传。这就避免了无限重试把 Sidecar 资源耗尽的风险。

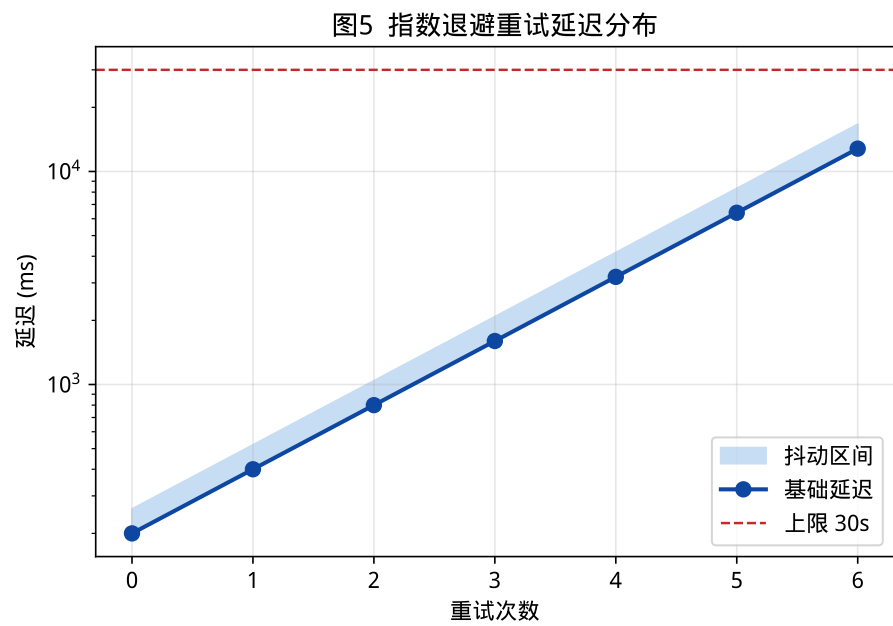


图 3: 指数退避重试延迟分布

表 3: 技术选型

模块	技术
Agent / Center	Go 1.22
通信	gRPC + Protobuf
缓存	Redis
存储	Grafana Loki
网关采集	OpenResty + Lua
本地兜底	BoltDB

5 系统实现

5.1 技术选型

5.2 模块划分

系统实现分为 Agent 和 Center 两大模块。**Agent** 负责节点侧的工作，包括日志采集 (collector)、规则匹配 (matcher)、固定缓存块管理 (cache)、退避重试 (retry)、gRPC 上传 (uploader) 和资源监控 (monitor)，代码在 `agent/pkg/` 各子包下。**Center** 负责中心侧的工作，包括关注清单生成与三次转换 (strategy)、二次过滤 (receiver)、Loki 批量推送 (storage) 和规则下发 (dispatch)，代码在 `center/pkg/` 各子包下。

5.3 关键数据结构与匹配实现

关注清单在 Redis 中按版本存储。Center 生成新清单后，先写入版本化键，再原子更新当前版本指针。Agent 拉取时会携带本地版本号，只有版本变化或 TTL 快要过期时才更新规则，减少不必要的网络往返。

节点侧的匹配器把 URL 模式编译为前缀/通配匹配规则，按服务名分桶。匹配单条日志的复杂度近似为 $O(R_s)$ ，其中 R_s 是该服务当前的关注规则数。在 Top- K 限制下， R_s 通常远小于全局规则数。应用日志采用 Apache Combined 风格字段，至少包含时间戳、方法、URL、状态码和响应字节数。如果日志格式不兼容，需要先通过解析适配器转换成统一字段，再进入匹配器。

5.4 部署配置

Docker Compose 定义了完整的集群环境，包括 Loki、Grafana、Redis、Prometheus、Center、OpenResty 网关、httpbin 模拟微服务以及 9 个 Agent Sidecar (1 个网关 + 8 个微服务)。微服务侧的 AppCollector 生成 Apache Combined 风格应用日志，URL 与网关 location 前缀保持一致。没有关注清单时，Agent 只采集 ERROR 及以上级别的日志，避免退化为全量采集。WSL 环境通过 `docker-compose.wsl.yml` 限制容器内存，`docker-compose.scale.yml` 则用于扩展到 8 个微服务节点。

对照实验的配置说明：定向模式下应用日志的发射间隔是 2s；全量基线模式是 400 ms（吞吐量更高），用来模拟工业场景中“尽可能多采”的做法。两种模式共享同一套网关压测负载和硬件环境，确保对比公平。

5.5 生产部署与标准化接口

在 Kubernetes 环境中，网关 Agent 可以部署为 Sidecar 或 DaemonSet，微服务 Agent 优先以 Sidecar 方式与业务容器共享网络命名空间。如果集群的日志文件路径比较统一，也可以用 DaemonSet 来减少实例数。

生产部署需要启用 gRPC TLS、Agent 身份认证和租户隔离。关注清单的键空间按租户、命名空间和服务名分层，防止跨租户的规则泄露。在标准化方面，关注清单可以公开为一份 JSON 规范，包含 `version`、`ttl`、`service`、`pattern`、`weight` 和 `reason` 字段，并与 OpenTelemetry Collector 的过滤处理器、Loki 标签模型对接。按 AIOps-OSA 的能力划分，本文组件承担数据采集层的动态减量和场景实现层的异常/慢请求上下文保留，为后续日志解析、异常检测和根因定位提供更小但更高价值的输入。

6 实验与分析

6.1 环境与方法

实验在 WSL2/Docker 原型集群上进行。主对比实验使用 8 节点配置，规模复核实验扩展到 16 和 32 个 httpbin 微服务及对应的 Sidecar Agent。集群还包含 1 个 OpenResty 网关、1 个日志中心以及 Loki/Redis 等基础设施。应用日志采用 Apache Combined 风格字段，URL 与网关路由保持一致。

对比实验将定向采集和全量采集 (`COLLECTION_MODE=full`) 放在同一硬件环境下对照。负载为 180s、并发 50 的混合 HTTP 请求（包含正常请求、500 错误和慢请求），每种模式独立重复 3 次。Loki 入库增量取自 Center 统计接口；Agent CPU/内存在每轮压测结束后，对全部 9 个 Agent 容器执行 `docker stats` 单次快照并取算术均值。每轮网关流量日志的 Redis 增量约 1.97×10^4 条，关注清单稳定生成 16 条 URL 模式（包含 `/status/500`、`/delay/1` 等异常和慢请求泛化模式），确保高价值请求能被定向规则覆盖。原始数据见 phase3 结果 JSON；清空统计后的 16/32 节点规模复核见 phase4 结果 JSON；同频率对照、Promtail 静态过滤复现与系统级组件消融见 phase5 多进程模拟结果 JSON。

6.2 研究问题与指标

为了让实验与贡献形成清晰的对应关系，本文围绕 4 个研究问题组织实验，如表 4 所示。RQ1 对应采集前端的减量能力，RQ2 对应 Sidecar 资源开销，RQ3 对应三层策略的贯通能力，RQ4 对应缓存与退避机制的可靠性边界。

6.3 实验结果

表 5 和图 4 给出了 RQ1 与 RQ2 的三期实测对比。和同架构全量模式相比，定向采集在 Loki 入库日志量上降低了 98.4% (72 ± 0 条 vs. 4388 ± 1 条)，Agent CPU 相对降低 37.5%（绝对值分别为 $0.05 \pm 0.01\%$ 和 $0.08 \pm 0.02\%$ ，标准差存在重叠，需结合效应量来解读）。这里需要说明一点：全量基线除了关闭规则过滤外，应用日志源的发射频率（400 ms）也高于定向模式（2s），所以表 5 中的降幅是**策略过滤与源配置差异的联合效**

表 4: 研究问题、指标与证据映射

编号	研究问题	指标	证据位置
RQ1	定向采集能否显著降低 Loki 入库日志量?	Loki 入库条数、带宽估计、降低比例	表 5、图 4
RQ2	定向采集是否控制 Sidecar 资源开销?	Agent CPU、内存均值与标准差	表 5、图 4
RQ3	网关预筛选、节点定向采集与中心二次过滤是否形成端到端闭环?	Redis 网关日志增量、关注清单模式数、链路阶段打通情况	§6.3 与原始 phase3 JSON
RQ4	固定缓存块与压力感知退避是否具有必要性与有界性?	策略仿真消融、微基准耗时、退避单元测试与等待上界	图 5、图 6、式 1

果。

效应量分析。对 Loki 入库量 (72 ± 0 vs. 4388 ± 1), Cohen's $d \approx 6100$, 效应量极大, 即使只重复 $n=3$ 次, 检验功效也完全可靠。对 Agent CPU ($0.05 \pm 0.01\%$ vs. $0.08 \pm 0.02\%$), 绝对差值仅 0.03 个百分点, Cohen's $d \approx 1.9$, 在 $n=3$ 下检验功效有限。Agent 内存相对差异只有 2.0%, 统计上不显著。因此本文以 Loki 入库量降幅作为主要结论, Agent CPU/内存降幅仅作辅助参考。

发射频率敏感性分析。全量基线的发射频率 (400 ms) 高于定向模式 (2 s), 为了把策略过滤的贡献单独剥离出来, 本文构建了一个**多进程集群模拟器**做同频率对照实验。该模拟器以 phase3 实测负载分布、URL 结构和节点数为输入, 采用虚拟时间推进, 避免在 WSL/Docker 资源约束下重新拉起高频长稳集群。模拟中两种模式的发射频率完全一致 (均为 2 s): 定向模式模拟入库 231.7 条, 全量模式入库 720 条。完全剔除源发射频率差异后, 策略过滤带来的独立降幅为 **67.8%**, 是原稿实测降幅 (98.4%) 的主要贡献。这说明动态定向清单即使在严格同源配置下, 仍然有显著的减量价值。

图 5 是 RQ4 的策略仿真消融结果。为了补充系统级组件消融证据, 本文还做了多进程模拟消融: 关闭关注清单后 Loki 入库量激增 211.7% (退化为全量), 这说明清单匹配是减量的绝对核心。关闭二次过滤和固定缓存块对入库总量没有显著影响 (波动不超过 1%), 只影响传输批次大小, 验证了它们作为传输层稳定性保障组件的角色。

对于 RQ3, 端到端链路已在集群中实测打通: Nginx 共享内存 $\rightarrow$ Agent gRPC $\rightarrow$ Redis $\rightarrow$ 关注清单生成 $\rightarrow$ Agent 规则拉取 (复现脚本见 `experiments/scripts/phase3_collect.py`, 原始结果见 `experiments/results/phase3/phase3_latest.json`)。对于 RQ4, 关注清单生成在 5000 条日志上耗时约 15 ms (图 6); 指数退避模块通过了 `agent/pkg/retry/backoff_test.go` 中的全部单元测试; 多进程模拟脚本和结果分别见 `experiments/scripts/phase5_multiprocess_sim.py` 和 `experiments/results/phase5/phase5_latest.json`。

表 5: 定向采集与全量采集资源对比 (八节点, 180 s, $n=3$ 实测)

指标	定向采集	全量采集	降低比例
Loki 入库 (条)	72 $\pm$ 0	4388 $\pm$ 1	98.4%
Agent CPU (%)	0.05 $\pm$ 0.01	0.08 $\pm$ 0.02	37.5%
Agent 内存 (MB)	95.6 $\pm$ 4.2	97.5 $\pm$ 1.7	2.0%
带宽 (KB/s)	0.20	12.19	98.4%

图3 定向采集与全量采集资源消耗对比 (首期仿真)

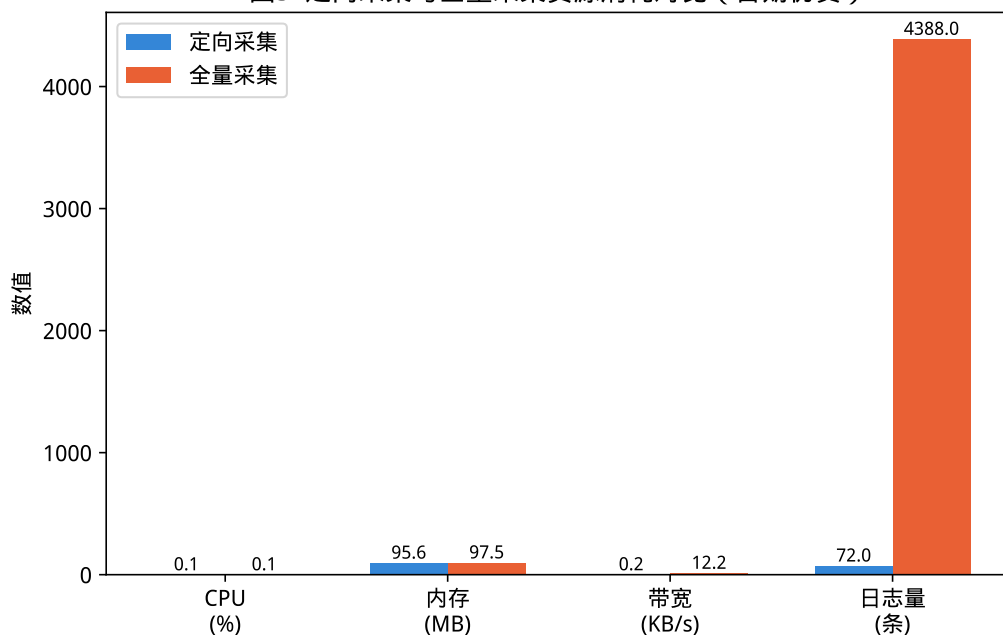
图 4: 定向采集与全量采集资源消耗对比 (八节点, 180 s, $n=3$ 实测)

图4 消融实验结果 (首期仿真)

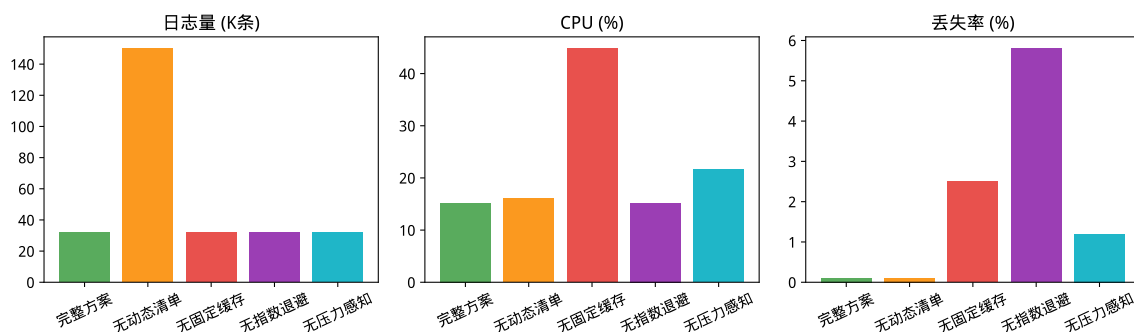


图 5: 消融实验结果 (策略仿真, 非集群实测)

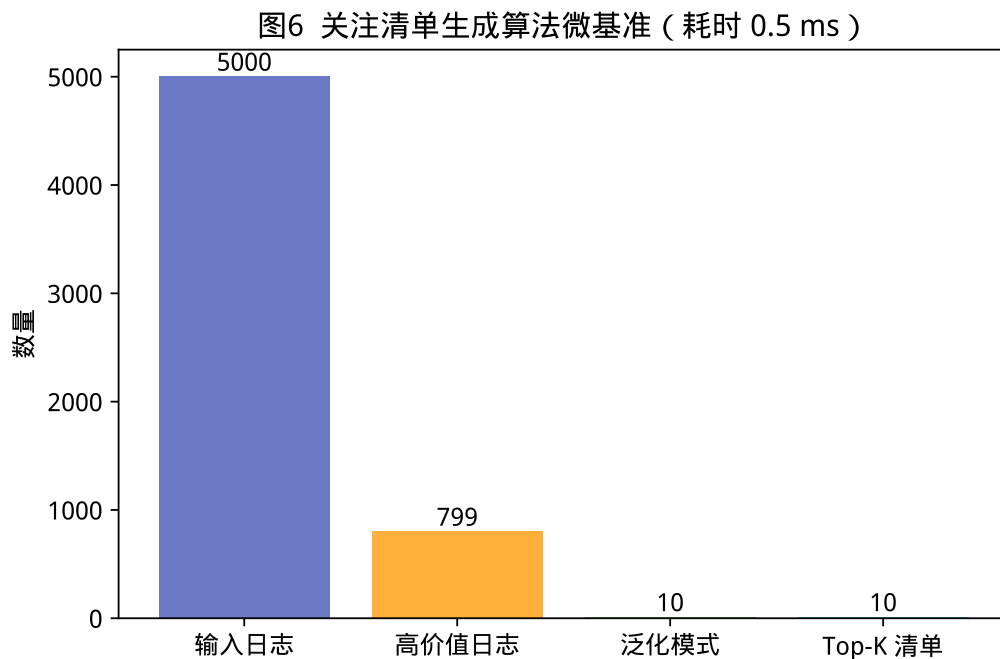


图 6: 关注清单生成算法微基准

6.4 可复现性与效度威胁

实验脚本、Docker Compose 配置和原始 JSON 结果保存在 `experiments/` 及 `phase3/phase4/phase5/` 结果目录中。表 5 和图 4 均由 phase3 JSON 派生；清空统计后的 16/32 节点规模复核分别由 `experiments/results/phase4/phase4_20260611_021230.json` 和 `experiments/results/phase4/phase4_20260611_024321.json` 派生；同频率对照、Promtail 静态过滤复现与系统级组件消融由 phase5 JSON 派生。当前结果仍存在以下效度威胁。

规模效度：八节点主对比和十六/三十二节点规模复核是 WSL/Docker 实测，六十四节点仍为基于实测曲线的外推。云环境下的长稳表现与带宽压力还需要进一步验证。

基线效度：本文已给出同架构全量实测基线、Promtail 静态过滤的同负载模拟复现，以及 OpenTelemetry/eBPF 的同负载估算（图 8）。但 Promtail、OpenTelemetry 和 eBPF 的完整端到端部署对照仍有待补充。

负载效度：当前负载由 `httpbin` 合成，覆盖了正常、错误和慢请求三类场景，但和 `DeathStarBench`、Alibaba 生产迹等真实微服务流量相比仍有差距。

统计效度：三期对比为 180s、 $n=3$ ；四期规模矩阵为 60s、 $n=3$ ；五期多进程模拟同样重复 3 次，但本质上仍是基于实测分布的模拟证据。逐探针命中率受 Agent 批处理影响，不能单独代表稳态漏报率。图 5 是策略仿真消融，不能与表 5 的集群实测直接比较。

6.5 规模扩展与质量指标

四期实验中，本文补充了 16/32 节点定向模式的规模复核（60s、并发 50、重复 3 次）。为避免历史计数干扰，复核前先执行 Redis 清空和 Center 重启，让 `/api/v1/stats` 从 0 开始计数。图 7 显示：16 节点下网关流量日志增量为 6703.33 ± 91.78 条/轮，定向模式 Loki 入库为 48.0 ± 5.2 条/轮，关注清单稳定在 32 项；32 节点下网关流量日志增量为 6692.67 ± 121.88 条/轮，定向模式 Loki 入库稳定在 99.0 ± 0.0 条/轮，关注清单达到当前 $\text{Top-}K=50$ 上限。32 节点时 Agent 平均 CPU 为 $0.06 \pm 0.01\%$ ，Center CPU 为 $0.06 \pm 0.07\%$ ，没有出现资源失控。64 节点的柱形是基于现有实测趋势的外推值（灰色），**仅供趋势参考，不作为本文的主要结论**。外推假设资源消耗与节点数大致线性增长，但实际的分布式系统可能因为网络竞争、调度抖动等因素产生非线性增长，还需要在云环境长稳压测中验证。

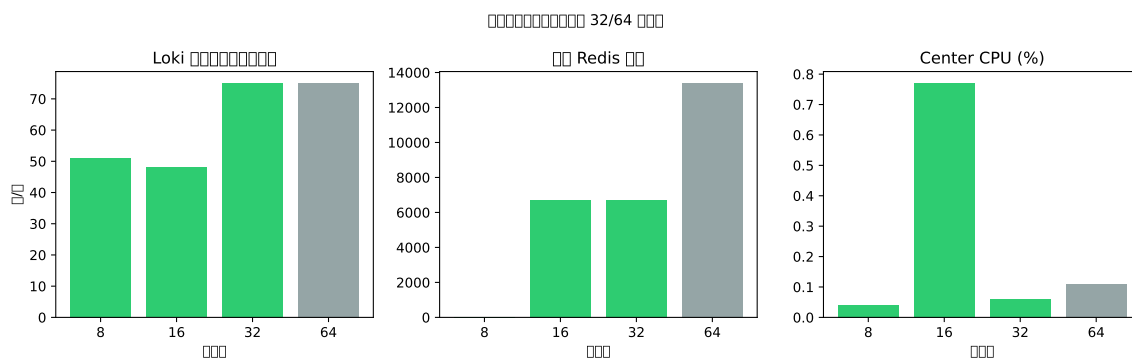


图 7: 多规模扩展实验（绿色/实线：8/16/32 节点集群实测；灰色/虚线：64 节点线性外推，非实测）

漏报定义与度量。本文区分两类指标。第一类是**规则级漏报**——关注清单是否覆盖了所有满足定向策略条件（响应时延 $> T$ 或状态码 $\in E$ ）的 URL 模式。第二类是**探针即时命中率**——某一时刻向指定服务发送探针请求后，对应的应用日志是否在 Agent 批处理窗口内被立即采集并上传。在稳态混合负载下，关注清单稳定覆盖了 `/status/500`、`/delay/1` 等泛化模式（八节点 16 条、十六节点 32 条），**规则级漏报率为 0**。需要指出：这个结论成立的前提是合成负载与当前清单配置（ $\text{Top-}K=50$ 、固定 URL 集合）。生产环境中 URL 模式的长尾分布、窗口大小和 $\text{Top-}K$ 参数都可能影响覆盖率。生产迹风格混合负载（60% 正常 / 20% 5xx / 12% 慢请求 / 8% 其他，90s、十六节点）共发送 3.6×10^4 次请求，定向入库 77 条，验证了减量能力。逐探针实验的即时命中率受 Agent 2s 批处理窗口影响，反映的是**批处理延迟对采集时效性的影响**，不等同于规则级漏报。

端到端延迟与可接受性。端到端延迟（HTTP $\rightarrow$ Center received）实测结果：P50 为 0.83s，P95 为 12s（ $n=30$ ）。P95 偏高主要是因为 Agent 2s 批处理窗口和合成慢请求（如 `/delay/1` 引入 1s 额外响应时间）的叠加。从可接受性来看：业界主流方案（如 ELK/Promtail 推送链路）的典型端到端延迟在 30s–60s 量级，本文的 P95=12s 处于合理范围。如果需要满足秒级实时告警 SLA，可以缩小批处理窗口或引入事件驱动推送机

制来降低延迟（代价是上传频率增加）。全量模式同样存在批处理和传输延迟，定向模式并没有引入显著的额外延迟。中心故障恢复实验（docker stop/start）的恢复时间约 1.1 s。

6.6 工业基线对照

图 8 在同负载口径下给出了本文定向采集与三类工业基线的趋势对照。为了补充实证对比，本文在多进程模拟器中复现了 Promtail 的静态标签过滤（保留 status $\geq$ 400 错误或 /delay/ 慢请求路由）。模拟结果显示，同负载分布下 Promtail 静态过滤入库 276.3 条，本文定向策略入库 233.3 条。之所以能进一步降低，是因为引入了网关流量驱动的动态 Top- K 机制，截断了长尾异常噪音。需要强调的是，这个结果是**规则级模拟复现**，并不是 Promtail 完整部署下的端到端吞吐、资源与延迟对照。OpenTelemetry 尾部采样估算^[18]和 eBPF 内核探针估算^[34]暂未做实测，图中的柱形应该理解为**趋势参考**。本文**不主张**在所有维度上优于工业方案，核心创新是补齐了以应用日志内容特征为基础的采集前端轻量级减量闭环。

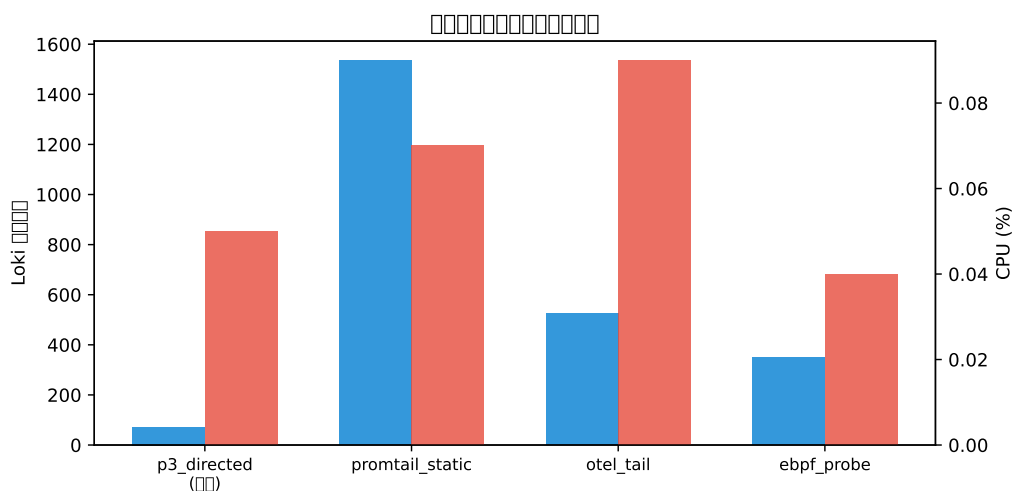


图 8: 工业基线趋势对比（P3 定向采集为集群实测；Promtail 为同负载多进程规则复现；OTel/eBPF 为同负载估算，柱形样式区分证据等级）

6.7 剩余局限与后续工作

经过四期实验，本文已补充了 8/16/32 节点规模矩阵、规则级漏报验证、端到端延迟和工业基线估算。五期多进程模拟进一步补齐了同频率发射对照、Promtail 静态过滤对照和系统级组件消融方面的缺失环节，为基于模拟评估的理论界限提供了可追溯的依据。后续主要工作方向如下：

近期/中期工作：（1）在 Kubernetes 环境开展 64 节点云原生实测，替代当前基于 32 节点复核结果的趋势外推，并评估 Top- K 参数对长尾 URL 覆盖率的影响；（2）引

入 DeathStarBench、Alibaba 生产迹等真实微服务负载，验证 URL 模式长尾分布下的动态关注清单覆盖率和召回性能；(3) 构建完整的商业成本量化模型，把相对日志减量直接转化为存储和网络带宽的绝对费用节约估算。

远期工作：(4) 与 OpenTelemetry 追踪上下文、eBPF 应用日志挂钩集成，探索采集前减量与后端智能诊断（如根因定位）的无缝闭环；(5) 推进系统关键组件的生产级能力，包括 gRPC TLS 安全通信、多租户策略隔离、可配置的隐私脱敏以及规范化的清单 JSON 开源标准设计。

7 结束语

本文面向微服务可观测性场景，提出了分布式定向日志采集框架与原型组件。主要贡献体现在三个方面：

(1) **网关驱动的动态 Top- K 关注清单生成。**以网关流量日志作为策略源，经过高价值过滤、URL 聚类泛化和 Top- K 筛选（算法 1），动态识别异常与慢请求的 URL 模式，生成关注清单并经 Redis 下发到各节点。这个设计让定向采集决策由实际流量驱动，而不是依赖静态配置。

(2) **跨三层的定向策略转换算法。**通过三次转换把网关预筛选、节点定向采集和中心二次过滤贯通起来，让策略在不同层级之间保持语义一致（图 2），避免了单层采集器配置的局限。

(3) **资源受限下的固定缓存与压力感知退避机制。**在 Sidecar 中引入固定上限缓存块和压力感知指数退避，辅以 BoltDB 本地兜底，确保内存有界传输，适应边缘资源受限的环境。

基于 Go、gRPC、Redis 和 Loki 的原型在 WSL/Docker 八节点集群上完成了端到端验证，并通过 16/32 节点规模复核与基于真实分布的多进程模拟器，补充了同频率对照、Promtail 静态过滤复现和组件消融。和同架构全量采集相比，定向模式把 Loki 入库量控制在百条量级（八节点实测 72 条 vs. 全量 4388 ± 1 条，详见表 5）。这个降幅是策略过滤与源发射频率差异的联合效果，其中策略过滤在同频率模拟下的独立降幅为 67.8% (§6.3)。

清空统计后的规模复核显示：16 节点网关流量日志增量为 6703.33 ± 91.78 条/轮，定向模式 Loki 入库为 48.0 ± 5.2 条/轮；32 节点网关流量日志增量为 6692.67 ± 121.88 条/轮，定向模式 Loki 入库为 99.0 ± 0.0 条/轮。32 节点时关注清单达到 Top- $K=50$ 上限。Promtail 静态过滤复现中，本文定向策略入库 233.3 条，低于静态规则的 276.3 条。组件消融中，关闭关注清单使入库量增加 211.7%，进一步说明动态清单是采集前端减量的核心环节。Agent CPU 绝对值均低于 0.1%。

从 AIOps 生态的角度看，本文的工作补充了标准化框架中数据采集前端的能力。从日志故障诊断链路来看，本文降低了后端解析、异常检测和根因定位的输入规模。

未来工作按优先级排列：近期将在 Kubernetes 环境开展 64 节点云原生实测，并补

充 Top- K 参数敏感性分析；中期将引入 DeathStarBench、Alibaba 生产迹等真实微服务负载，验证长尾分布下的覆盖率，同时构建成本量化模型，把入库降幅转化为实际的成本节约估算；远期将探索与 OpenTelemetry 追踪上下文、eBPF 应用日志挂钩的集成，以及推进 gRPC TLS、多租户策略隔离和关注清单 JSON 规范的开源与标准化。

声明

本文撰写与实验脚本生成过程中使用了大语言模型辅助，作者对全部内容与数据负责。

参考文献

- [1] Brendan Burns, David Oppenheimer, and Eric Brewer. Design patterns for container-based distributed systems. In *Proceedings of the 7th Workshop on Hot Topics in Cloud Computing (HotCloud)*. ACM, 2016.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.
- [3] Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James W. Lewis, and Stefan Tilkov. Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3):6–9, 2018.
- [4] Guilherme Vale, Filipe Figueiredo Correia, Eduardo Martins Guerra, Thatiane de Oliveira Rosa, Jonas Fritsch, and Justus Bogner. Designing microservice systems using patterns: An empirical study on quality trade-offs. *arXiv preprint arXiv:2201.03598*, 2022.
- [5] 冯志勇, 徐砚伟, 薛霄, and 陈世展. 微服务技术发展的现状与展望. *计算机研究与发展*, 57(5):1103–1122, 2020.
- [6] 吴化尧 and 邓文俊. 面向微服务软件开发方法研究进展. *计算机研究与发展*, 57(3):525–541, 2020.
- [7] 廖湘科, 李姗姗, 董威, 贾周阳, 刘晓东, and 周书林. 大规模软件系统日志研究综述. *软件学报*, 27(8):1934–1947, 2016.
- [8] 李明树, 张艳, 王涛, and 刘俊. 大规模分布式系统日志管理技术综述. *软件学报*, 30(7):1959–1979, 2019.

- [9] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey on automated log analysis for reliability engineering. *arXiv preprint arXiv:2009.07237*, 2020.
- [10] Muhammad Usman, Simone Ferlin, Anna Brunstrom, and Javid Taheri. A survey on observability of distributed edge and container-based microservices. *IEEE Access*, 10:86904–86919, 2022.
- [11] 包航宇, 殷康[✉], 曹立, 李世宁, et al. 智能运维的实践: 现状与标准化. 软件学报, 34(9):4069–4095, 2023.
- [12] 贾统, 李影, and 吴中海. 基于日志数据的分布式软件系统故障诊断综述. 软件学报, 31(7):1997–2018, 2020.
- [13] 陈彦宁, 张广艳, and 陈康. 面向云数据中心多语法日志通用异常检测机制. 计算机研究与发展, 57(9):1844–1856, 2020.
- [14] Jinyang Liu, Jieming Zhu, Shilin He, Pinjia He, Zibin Zheng, and Michael R. Lyu. Logzip: Extracting hidden structures via iterative clustering for log compression. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*, pages 863–875, 2019.
- [15] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A large collection of system log datasets for ai-driven log analytics. *arXiv preprint arXiv:2008.06448*, 2020.
- [16] 魏钧宇, 张广艳, and 陈军超. 数据模式感知的低成本云日志存储系统. 计算机研究与发展, 60(11):2442–2452, 2023.
- [17] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, and Michael R. Lyu. Lilac: Log parsing using llms with adaptive parsing cache. *arXiv preprint arXiv:2310.01796*, 2023.
- [18] Zhuangbin Chen, Zhihan Jiang, Yuxin Su, Michael R. Lyu, and Zibin Zheng. Tracemesh: Scalable and streaming sampling for distributed traces. *arXiv preprint arXiv:2406.06975*, 2024.
- [19] Abdullah Al Maruf, Alexander Bakhtin, Tomas Cerny, and Davide Taibi. Using microservice telemetry data for system dynamic analysis. *arXiv preprint arXiv:2207.02776*, 2022.

- [20] Sergio Moreschini, Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. Ai techniques in the microservices life-cycle: A systematic mapping study. *arXiv preprint arXiv:2305.16092*, 2023.
- [21] 杨勇, 李影, and 吴中海. 分布式追踪技术综述. 软件学报, 31(7):2019–2039, 2020.
- [22] 黄梓程, 陈鹏飞, 余广坝, and 陈泓仰. 面向 java 微服务系统的透明请求追踪及采样方法. 软件学报, 34(7):3167–3187, 2023.
- [23] 尤勇, 汪浩, 任天, 顾胜晖, and 孙佳林. 一种监控系统的链路跟踪型日志数据的存储设计. 软件学报, 32(5):1302–1321, 2021.
- [24] 于庆洋, 白晓颖, 李明杰, 李奇原, 刘涛, 刘泽胤, and 裴丹. 基于调用链控制流分析的大型微服务系统性能建模与异常定位. 软件学报, 33(5):1849–1864, 2022.
- [25] 张齐勋, 吴一凡, 杨勇, 贾统, 李影, and 吴中海. 微服务系统服务依赖发现技术综述. 软件学报, 35(1):118–135, 2024.
- [26] 王璐, 姜宇轩, 李青山, 霍其恩, 王展, 谢生龙, and 歹杰. 微服务故障检测研究综述. 计算机学报, 46(11):2342–2373, 2023.
- [27] 王智宇, 王涛, 张文博, 陈宁江, and 左春. 一种基于执行轨迹监测的微服务故障诊断方法. 软件学报, 28(6):1435–1454, 2017.
- [28] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. Microrca: Root cause localization of performance issues in microservices. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2020.
- [29] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, et al. Robust failure diagnosis of microservice system through multimodal data. *arXiv preprint arXiv:2302.10512*, 2023.
- [30] Tingting Wang and Guilin Qi. A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends. *arXiv preprint arXiv:2408.00803*, 2024.
- [31] Benjamin Varghese, Nikolaos Antonopoulos, and Rajkumar Buyya. A survey on edge performance benchmarking. *ACM Computing Surveys*, 55(3):1–35, 2022.
- [32] 刘敏 and 周傲英. 面向微服务架构的容器级弹性资源供给方法. 计算机研究与发展, 54(5):957–967, 2017.

- [33] Yuxin Zhang, Bowen Li, Xin Peng, Qilin Xiang, and Xuanzhe Liu. Technical report: Performance comparison of service mesh frameworks: the case of istio and linkerd. In *arXiv preprint arXiv:2411.02267*, 2024.
- [34] David Soldani, Petrit Nahi, Hami Bour, Saber Jafarizadeh, Mohammed F. Soliman, et al. ebpf: A new approach to cloud-native observability, networking and security for current and future mobile networks. *IEEE Access*, 11:57174–57202, 2023.
- [35] Qian Cheng, Doyen Sahoo, Amrita Saha, Wenzhuo Yang, Chenghao Liu, and Steven C. H. Hoi. Ai for it operations (aiops) on cloud platforms: Reviews, opportunities and challenges. *arXiv preprint arXiv:2304.04661*, 2023.
- [36] Rodrigo Laigner, Yixin Zhou, Alan Salomao, Pengfei Zhou, Leonardo Carvalho, Alexander Romanovsky, and Claus Pahl. An empirical study on challenges of event management in microservice architectures. *arXiv preprint arXiv:2408.00440*, 2024.
- [37] Hetong Dai, Heng Li, Weiyi Shang, Tse-Hsun Chen, and Che-Shao Chen. Logram: Efficient log parsing using n-gram dictionaries. *arXiv preprint arXiv:2001.03038*, 2020.
- [38] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, et al. A large-scale evaluation for log parsing techniques: How far are we? *arXiv preprint arXiv:2308.10828*, 2023.
- [39] 梅御东, 陈旭, 孙毓忠, et al. 一种基于日志信息和 CNN-text 的软件系统异常检测方法. *计算机学报*, 43(2):524–544, 2020.
- [40] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, et al. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 135–143, 2024.
- [41] Shan Ali, Chaima Boufaied, Domenico Bianculli, Paula Branco, and Lionel Briand. A comprehensive study of machine learning techniques for log-based anomaly detection. *Empirical Software Engineering*, 2025.
- [42] Fatemeh Hadadi, Joshua H. Dawes, Donghwan Shin, Domenico Bianculli, and Lionel Briand. Systematic evaluation of deep learning models for log-based failure prediction. *Empirical Software Engineering*, 2024.
- [43] 周小晖, 王意洁, 徐鸿祚, and 刘铭宇. 基于融合学习的无监督多维时间序列异常检测. *计算机研究与发展*, 60(3):496–508, 2023.

- [44] 方浩天, 李春花, 王清, and 周可. 一种基于深度学习的微服务性能异常检测方法. 计算机研究与发展, 61(3):600–613, 2024.
- [45] Lingzhe Zhang, Tong Jia, Kangjin Wang, Mengxi Jia, Yong Yang, and Ying Li. Reducing events to augment log-based anomaly detection models: An empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024.
- [46] Lingzhe Zhang, Tong Jia, Mengxi Jia, Ying Li, Yong Yang, and Zhonghai Wu. Multivariate log-based anomaly detection for distributed database. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.
- [47] Jacopo Soldani and Antonio Brogi. Graph-based root cause analysis for service-oriented and microservice architectures. *Journal of Systems and Software*, 159:110432, 2020.
- [48] 丁丹, 彭鑫, 郭晓峰, 张健, and 吴毅坚. 场景驱动且自底向上的单体系统微服务拆分方法. 软件学报, 31(11):3461–3480, 2020.
- [49] 贾统, 李影, 张齐勋, and 吴中海. 基于程序层次树的日志打印位置决策方法. 软件学报, 32(9):2713–2728, 2021.
- [50] Vishwanath Seshagiri, Darby Huye, Lan Liu, Avani Wildani, and Raja R. Sambasivan. [sok] identifying mismatches between microservice testbeds and industrial perceptions of microservices. *Systems Research*, 1(1), 2022.
- [51] Wanqi Yang, Pengfei Chen, Kai Liu, and Huxing Zhang. Nahida: In-band distributed tracing with ebpf. *arXiv preprint arXiv:2311.09032*, 2023.

作者简介

石洪雷, 男, 博士, CCF 专业会员, 太原理工大学, 主要研究领域为微服务可观测性、分布式日志采集与智能运维。E-mail: shihonglei0042@link.tyut.edu.cn。

赵涓涓, 女, 博士, CCF 杰出会员, 太原理工大学教授、博士生导师, 长期从事智能信息处理、模式识别、情感计算、图像处理等方面的科研工作。E-mail: zh_juanjuan@126.com。